計算機アプリ コード解説

仕組み解説PDFとは別冊。実際のコードを上から読んでいく版です。

0. ファイルの役割

ファイル	何をしているか
`src/types.ts`	型（門番）の定義
`src/utils/calculate.ts`	四則演算だけ
`src/components/Display.tsx`	画面表示
`src/components/Button.tsx`	ボタン見た目とクリック
`src/App.tsx`	司令塔（state と処理）

1. types.ts — 型

src/types.ts

```
export type Operator = "+" | "−" | "×" | "÷";
export type Digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
export type CalculatorButton = "digit" | "operator";
```

> **読み方:** `Operator` は「この4文字だけ許す」という意味。
> `|` は「または」。`"*"` や `"/"` はここに無いので TypeScript がエラーにする。

2. calculate.ts — 計算だけ

src/utils/calculate.ts

```
export function calculate(
  a: number,
  b: number,
  operator: Operator
): number {
  switch (operator) {
    case "+": return a + b;
    case "−": return a - b;
    case "×": return a * b;
    case "÷": return a / b;
    default: throw new Error("Invalid operator");
  }
}
```

部分	意味
`a`	前の数（previousValue）
`b`	今の画面の数
`operator`	"+" など
`: number`	戻り値も数値

> 例: `calculate(7, 8, "+")` → `15`
> ボタンの見た目は知らない。計算だけ担当する。

3. Display.tsx — 渡された文字を出す

src/components/Display.tsx

```
type DisplayProps = {
  value: string;
  className?: string;
};

export function Display({ value, className }: DisplayProps) {
  return <div className={className}>{value}</div>;
}
```

> `value` は App から来る `displayValue`。
> Display 自身は計算しない。「渡された文字列を表示する」だけ。

4. Button.tsx — 見た目とクリック

src/components/Button.tsx（要点）

```
type ButtonProps = {
  label: string;
  variant: CalculatorButton; // "digit" | "operator"
  onClick?: () => void;
};

export function Button({ label, variant, className, onClick }: ButtonProps) {
  const colors = {
    digit: "bg-zinc-400 text-black",
    operator: "bg-orange-500 text-white",
  };
  const colorClass = colors[variant];
  return (
    <button type="button" className={...} onClick={onClick}>
      {label}
    </button>
  );
}
```

> `label` … ボタンに書く文字（"7" や "+"）
> `variant` … 色の種類
> `onClick` … 押されたとき App が渡した関数を実行

5. App.tsx — 司令塔

5-1. import

```
import { useState } from "react";
import type { Operator } from "./types.ts";
import { calculate } from "./utils/calculate.ts";
import { Display } from "./components/Display";
import { Button } from "./components/Button.tsx";
```

> `import type` は型だけ取り込む書き方。
> `calculate` は実行時に使う本物の関数。

5-2. useState（4つのメモ）

```
const [displayValue, setDisplayValue] = useState("0");
const [previousValue, setPreviousValue] = useState<number | null>(null);
const [operator, setOperator] = useState<Operator | null>(null);
const [waitingForOperand, setWaitingForOperand] = useState(false);
```

コード	読み方
`useState("0")`	最初の画面は "0"。左が今の値、右が書き換え関数
`number | null`	数値か、まだ無い（null）
`Operator | null`	演算子か、まだ選んでいない
`useState(false)`	最初は「次の数字待ち」ではない

5-3. pushNumber — 数字ボタン

```
function pushNumber(digit: string) {
  if (waitingForOperand) {
    setDisplayValue(digit === "0" ? "0" : digit);
    setWaitingForOperand(false);
    return;
  }

  setDisplayValue((current) => {
    if (current === "0") {
      return digit === "0" ? "0" : digit;
    }
    return current + digit;
  });
}
```

> **上の if:** 演算子の直後なら、画面を新しい1桁だけにする（"12" にしない）
> **下の setDisplayValue((current) => …):** React が今の画面を `current` として渡す。
> `current + digit` は文字列連結。"1" + "2" → "12"

> ボタン側はこう呼ぶ: `onClick={() => pushNumber("7")}`
> ここで渡した `"7"` が、関数の引数 `digit` になる。

5-4. chooseoperator — 演算子ボタン

```
function chooseoperator(op: Operator) {
  if (
    operator !== null &&
    previousValue !== null &&
    waitingForOperand === false
  ) {
    const result = calculate(previousValue, Number(displayValue), operator);
    setDisplayValue(String(result));
    setPreviousValue(result);
  } else {
    setPreviousValue(Number(displayValue));
  }
  setOperator(op);
  setWaitingForOperand(true);
}
```

> **if の中（連続計算）**：すでに演算中なら、先に今の計算をして結果を覚える
> **else:** 初めて演算子を押した → 今の画面を前の数にする
> 最後に、押した演算子を覚え、次の数字待ちにする

例: `onClick={() => chooseoperator("+")}` → 引数 `op` は `"+"`

5-5. handleEquals — = ボタン

```
function handleEquals() {
  if (operator === null || previousValue === null) {
    return; // まだ計算できない
  }
  const result = calculate(previousValue, Number(displayValue), operator);
  setDisplayValue(Number.isFinite(result) ? String(result) : "Error");
  setOperator(null);
  setPreviousValue(null);
  setWaitingForOperand(true);
}
```

> `Number(displayValue)` … 画面の文字 "8" を数値 8 にする
> `String(result)` … 数値 15 を文字 "15" にして画面へ
> 最後の3行 … 計算メモを消し、次の数字で新しい計算を始められるようにする

> `handleEquals` に引数は無い。`=` は計算の種類ではなく「実行ボタン」。
> 演算子はすでに `operator` state に入っている。

5-6. AC — 全部リセット

```
onClick={() => {
  setDisplayValue("0");
  setPreviousValue(null);
  setOperator(null);
  setWaitingForOperand(false);
}}
```

> 画面だけ戻すと、裏の計算メモが残る。4つまとめて初期値に戻す。

5-7. JSX — 画面へのつなぎ

```
<Display value={displayValue} />

<Button label="7" variant="digit"
  onClick={() => pushNumber("7")} />

<Button label="+" variant="operator"
  onClick={() => chooseoperator("+")} />

<Button label="=" variant="digit"
  onClick={() => handleEquals()} />
```

> `value={displayValue}` … 波括弧の中は JavaScript の値
> 数字ボタン → `pushNumber`
> 演算子 → `chooseoperator`
> = → `handleEquals`

6. コードで追う：7 + 8 =

操作	呼ばれる関数	主な変化
7	`pushNumber("7")`	displayValue → "7"
+	`chooseoperator("+")`	previousValue → 7 / operator → "+" / waiting → true
8	`pushNumber("8")`	waiting が true なので displayValue → "8"（末尾に足さない）
=	`handleEquals()`	calculate(7, 8, "+") → 15 → 画面 "15"、演算メモをクリア

コードを読む順番のおすすめ
① types → ② calculate → ③ Display / Button → ④ App の useState → ⑤ pushNumber → ⑥ chooseoperator → ⑦ handleEquals → ⑧ JSX の onClick